National College of Ireland

Project Submission Sheet

Student Name:	Aatmaja Kalpesh Karbhari, Mrunal Sunil Shende, Vishal Verma, Meenakshi
Student ID:	24107620, 24245909, 24275824, 24235415
Programme:	MSc in Cloud Computing **Year:** 2026
Module:	Cloud Machine learning
Lecturer:	Mr.Vikas Tomer
Submission Due Date:	13th April 2026,
Project Title:	Online Delivery Time Prediction
Word Count:	3803

Academic Integrity Certificate Verification Code y2xTgCHENv, YVRTLiB4kG, 30WFRi2HcY, Q6lveHvE6b

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature: ……Aatmaja,Mrunal,Vishal,Meenakshi……

Date: ……11-04-2026……

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).

2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

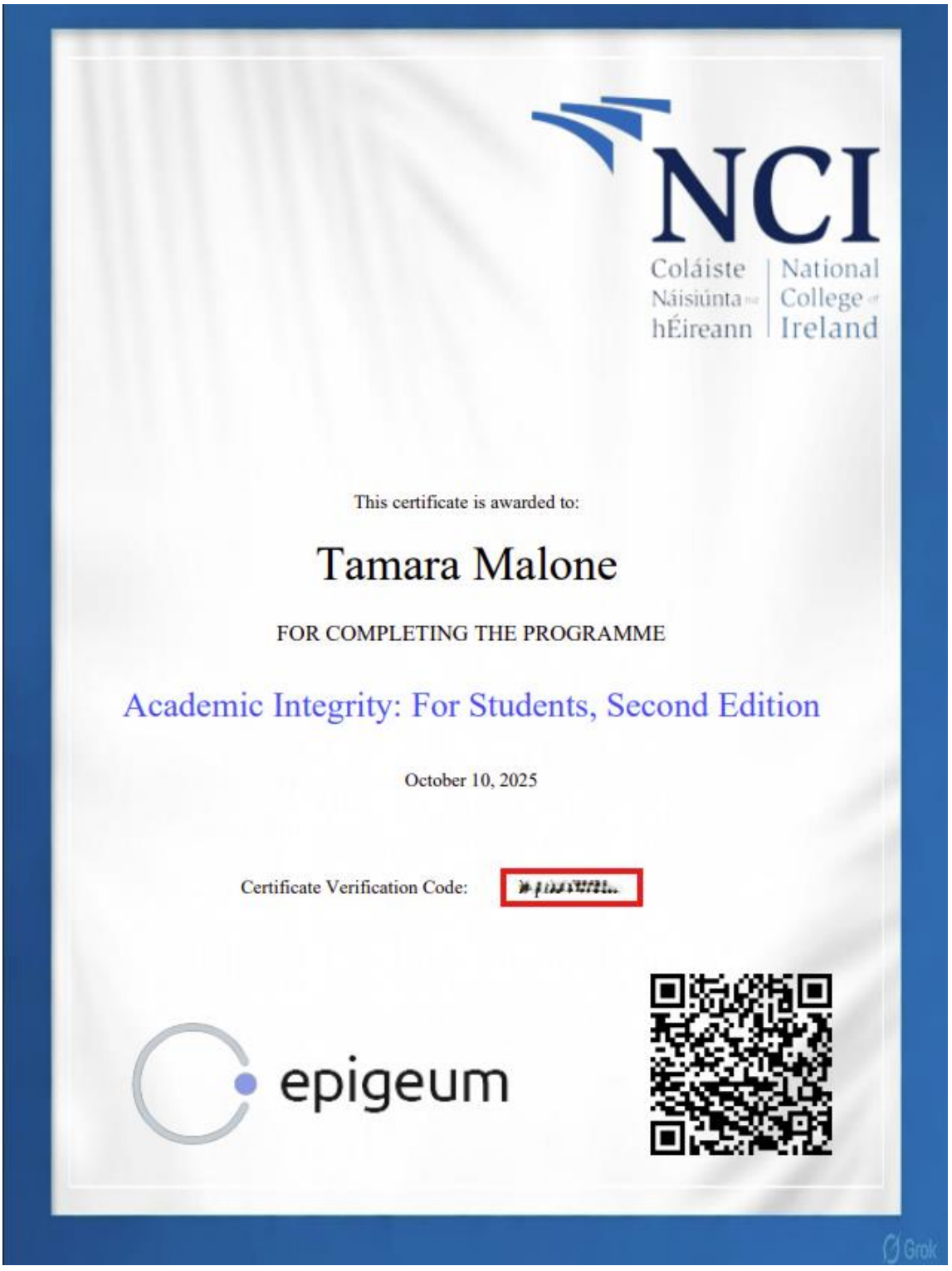
This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field



Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents
Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

Online Delivery Time Prediction

Aatmaja Kalpesh Karbhar
Cloud Computing
National College Of Ireland
Dublin, Ireland
x24107620@student.ncirl.ie

Mrunal Sunil Shende
Cloud Computing
National College Of Ireland
Dublin, Ireland
x24245909@student.ncirl.ie

Vishal Verma
Cloud Computing
National College Of Ireland
Dublin, Ireland
x24275824@student.ncirl.ie

Meenakshi
Cloud Computing
National College Of Ireland
Dublin, Ireland
x24235415@student.ncirl.ie

Abstract*— Delivery time Plays a major role in customers statistics action especially in services like food delivery, ecommerce, and Courier logistics. However, predicting delivery time accurately is difficult because it depends on many changing factors such as distance, weather condition, and rider availability. This project focuses on building a machine learning based system that can estimate delivery time more reliably. We used the Random Forest algorithm because it handles mixed data, reduce overfitting and performs well with real world variations. The model was trained on the dataset containing key delivery features and evaluated using metrics such as MAE, RSME and R2. To make the system practical and accessible the entire pipeline was deployed on AWS using services like S3, Lambda, API gateway, CloudWatch and AWS Glue for data processing. The overall goal of this project is to support businesses in improving planning, reducing delays and offering more accurate delivery estimates to customers. This system shows that using cloud-based services can help make machine learning applications that are efficient and can handle a lot of work. The final system includes a way to compare the three machine learning models, a real-time prediction API, a way to monitor how well the system is working and a special dashboard that gives insights into how the delivery process works for online delivery platforms.

Keywords*— S3, Lambda, API gateway, CloudWatch, AWS Glue, Random Forest, linear regression and neural network.

I. INTRODUCTION

Fast and predictable delivery has become a basic expectation in today's world. Whether someone order food, groceries or a package, they want to know exactly when it will arrive. To businesses, an accurate estimate of delivery is not only a convenience; it has a direct bearing on customer trust, planning of operations, and efficiency in general. However, it remains difficult to forecast the time required to deliver, as it depends on a great number of real-world variables which are changing day by day out. Distance, weather, rider workload and order characteristics all influence how long a delivery will take.

In this project, we aim at solving these issues through machine learning that will optimize the prediction of delivery time. The dataset used in this project is a custom-created delivery data set design specifically for predicting delivery time each row in the data set represents one delivery order and each column represents of feature that affects how long the delivery takes. We analyzed delivery related data, identified the most important feature and trained a Random Forest model to estimate delivery duration. The choice to use random forest was based on its resistance to noisy data, ability to detect intricate patterns and the ability to provide consistent predictions.

The system was placed on Amazon Web Services to be a scalable and user-friendly system. Raw data is stored in S3, processed using AWS Glue and the trained model is hosted through AWS Lambda. API gateway allows the model to be accessed through a simple API and CloudWatch handles monitoring. There was a user-friendly dashboard that was designed to display predictions and model output in real time. By combining machine learning with cloud deployment this project demonstrates a practical solution that businesses can use to improve delivery accuracy, optimize operations and enhance customer experience. The main objective of the study is to determine whether machine learning algorithms, namely Linear Regression, Random Forest, and Neural Network, can be used to predict delivery times within a cloud-based infrastructure in a credible and dependable manner, and at the same time provide a viable and scalable prediction platform to the final consumers.

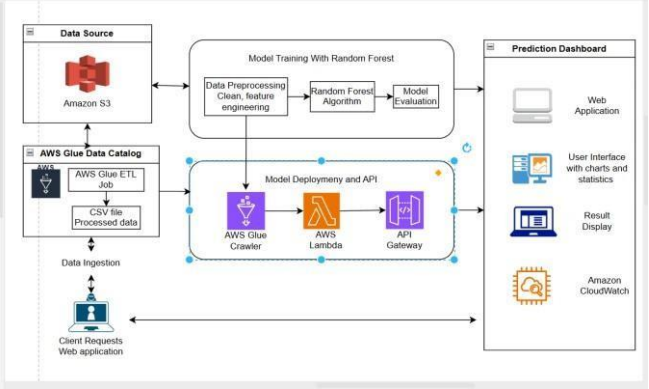


Fig. [1] Online delivery Time Prediction Architecture Diagram

II. RELATED WORK

The problem of precise delivery time has been studied thoroughly in logistics and food delivery, as well as in courier

services since an accurate estimation of time delivery maximizes consumer satisfaction and optimization of operations. Traditional regression methods have been overwhelmingly used in previous studies as they are simple and easy to interpret.

Linear regression can provide a clear mapping between the explanatory variables and the delivery time but the major limitation with the method is that it assumes only mostly linear relationships which may not wholly reflect the complex relationship that exists between weather, distance and the availability of riders in the real-world delivery context. In modern research, the appropriateness of tree- based models, like Random Forest, in the process of predicting delivery orders, has been emphasized. Their main strength lies in the ability to have non-linear associations, the heterogeneous features and the noisy and real-world data, and they are less vulnerable to overfitting as compared to single decision trees. However, random forest models are most likely to be less interpretable than linear regression and can be more computationally intensive.

They are therefore preferred where prediction accuracy is more important than that of model simplicity. Neural networks are also applied in situations where data is non-linear and complicated. Their power lies in the possibility to simulate complex relationships; however, they normally demand a lot of hyperparameters optimization, lengthy training time and careful choice of parameters. In addition, neural networks can be less suitable with smaller and structured data than tree-based techniques.
The neural networks were also taken into consideration in the current work as an alternative model to examine the possibility of improving predictive performance by increasing the complexity of the models. Another area of concern in previous literature relates to the use of cloud-based systems to implement machine learning systems. Services like AWS make the storage and processing of data, as well as deploying models and exposing APIs simple.

The advantages of cloud-based deployment concern the ability to scale up and accessibility, and constraints are concerned with the reliance on latency, monitoring, and integration of the services. Based on these considerations, the proposed project would make use of machine learning and AWS services to build not only a predictive model with high accuracy but also a working prototype that deployed to the final users.
This version is better than the earlier ones as it provides an explicit comparison of the following modelling paradigms: -

1) Simple yet constrained Linear Regression.

2) Random Forest: powerful but less understandable.

3) Neural Network: flexible but difficult to tune.

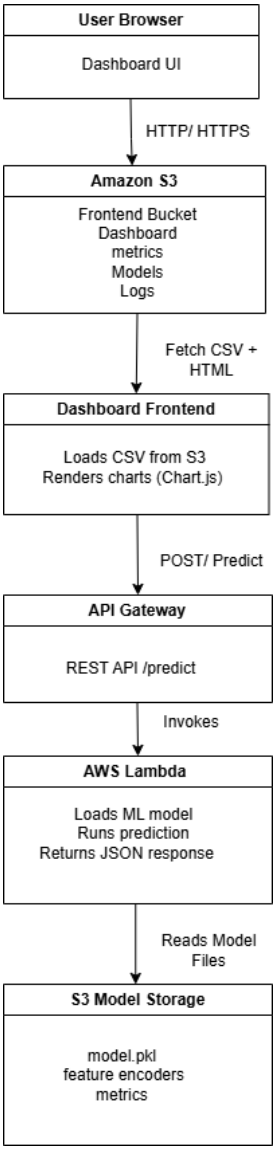


Fig. [2] Online Delivery Prediction Flow-Diagram

III. METHODOLOGY

The methodology of this project describes how the data was prepared, how the model was trained and how the entire system was deployed using AWS. The goal was to come up with an end-to-end pipeline which includes the entire data lifecycle, including raw inputs, to real-time predictions

1. Data Collection:

The dataset used in this project contains information related to deliveries, such as distance, weather conditions, number of riders, order value, and actual delivery time. These features were chosen because they directly influence how long a delivery takes. The raw data was stored in an Amazon S3 bucket.

2. Data Cleaning and Transformation:

Raw data do not always have all the values, they may also have inconsistent forms, or irregularities and this requires

pre-processing. To overcome this, AWS Glue that is made up of Crawler and an ETL job, was used.

- The Glue Crawler scanned the raw data and created a structure table in the Glue data catalos.
- The Glue ETL Job cleans the data by removing missing values converting data types and preparing the dataset for training. The cleaned data set was then saved back into S3 in a separate folder.

Feature	Count	Mean	Std	Min	25%	50%	75%	Max
Distance (km)	9922	4.24	2.14	0.5	2.45	4.22	5.99	12
Items	9922	10.62	5.65	1	6	11	15	20
Order Value	9922	783	408	50	444	787	1120	1500
Hour	9922	11.48	6.75	0	6	11	17	23
Weekend	9922	0.26	0.44	0	0	0	1	1
Peak Hour	9922	0.32	0.47	0	0	0	1	1
Riders Available	9922	15.52	8.5	1	8	15	23	30
Delivery Time (min)	9922	28.51	10.4	5	22	27.9	33.8	120

Table 1: Summary Statistics of Numerical Feature

3. *Feature Engineering:*

Before Training the model, we prepared the features this included:

- Converting categorical value into Numerical form
- feature standardization or scaling
- Selecting the most important features
- elimination of redundant or irrelevant attributes and phase out elimination of duplicates was done. This is a move that will play a critical role in making sure that the model absorbs information more effectively.

  This step helps the model learn more effectively.

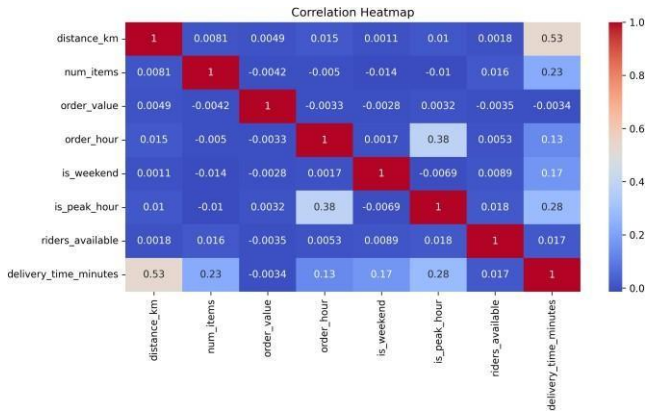


Fig. [3] Correlation Heatmap

4. *Model Selection:*

We chose the Random forest regressor because:

- The model supports non-linear association.
- It works well with mixed data types
- It reduces overfitting
- It provides stable predictions

  Random Forest boosts many decision trees to improve predictive accuracy because it averages the results of these decision trees.

5. *Model Training:*

The clean dataset was split into training and testing sets. Random Forest model was trained using the training data and then tested using an unknown test data. We tuned parameters such as:

- Number of trees
- Maximum depth
- Minimum samples per split

Training was done followed by the serialization of the model as a .pkl file and uploading it in S3.

6. *Model Deployment:*

- To facilitate the model, we implemented it with AWS
- AWS Lambda loads the model from S3 and performs predictions.
- API Gateway exposes a rest api endpoint that the granted can call.

7. *Dashboard Development:*

A simple web dashboard was created using HTML, CSS, JavaScript, Chart.js, and AOS animations.
The dashboard allows users to:

- Enter delivery details
- View predicted delivery time
- See model performance metrics
- Visualize charts and graphs

The dashboard is hosted on Amazon S3(static website hosting).

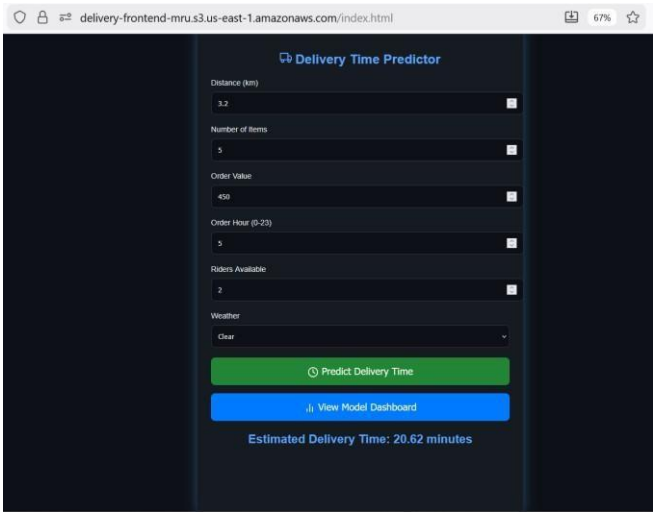


Fig. [4] Delivery Time Prediction Frontend

IV. EVALUATION

In this project the evaluation was done in several stages starting from checking the quality of the data set and

measuring the accuracy of Random Forest model and finally testing the entire AWS based deployment to see how the system behaves when real users interact with it. The goal of this evaluation was not only to measure numbers but also to understand the strength and weakness of the model and the overall pipeline the first part of the evaluation focus on the data set before training the model the dataset was check for missing values inconsistent entries and outliers this step is important because poor quality data can lead to poor quality prediction after cleaning the data using AWS blue the dataset became more structured and easier for the model to learn from the evaluation at this stage shows that the data set had a good variety of delivery scenarios including different distances weather conditions rider counts and order values the variety held the model learnt pattern more effectively.

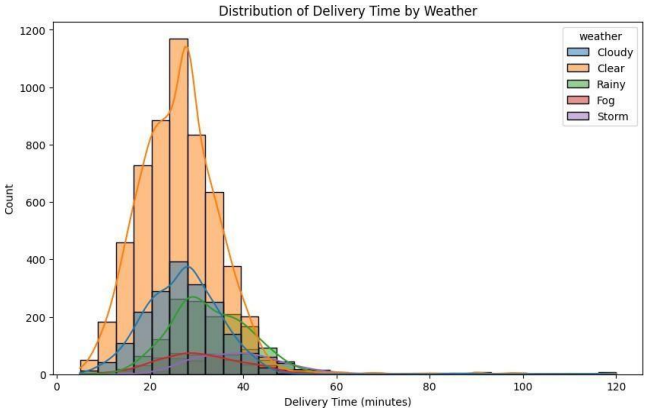


Fig. [5] Delivery Time by Weather

The next stage of evaluation involves the random forest model itself to understand how well the model predict delivery time the data set was divided into training and testing set the model was trained on one part of the data and tested on the other part that it had never seen before this helps major how well the model generalized a new data several evaluation matrix were used including mean

To test the performance of the model using data that the model was trained with, a random split of the dataset into training and testing was done. Random sampling eliminates selection bias, and both subsets will be a representative sample of a wide range of delivery conditions. The design is suitable in the case of the regression task since the target variable is a continuous variable (in this case), and the aim is the generalization of the models to new delivery records. In particular, the data was subdivided by 80 and 20 percent respectively creating a training and testing set respectively. Models were fitted using the training set and the testing set was only used to evaluate the end performance. Absolute error (MAE) root mean square error (RMSE) & R2 score this matrix gives different perspective on the model's performance. The reason Mean Absolute Error (MAE) was chosen was that it measures the mean amounts of the absolute changes of prediction measured in minutes and can be easily interpreted in a delivery scenario. Root Mean Square Error (RMSE) was added so that larger discrepancies get more weight since it is understood that grossly misleading estimates of delivery may result in loss of customer faith and a complex operational plan. Coefficient of Determination (R 2) was also given to show the percentage of variance in delivery time that was explained by the model.



Fig. [6] Algorithms Metrics Table



Fig. [7] RMSE Comparison

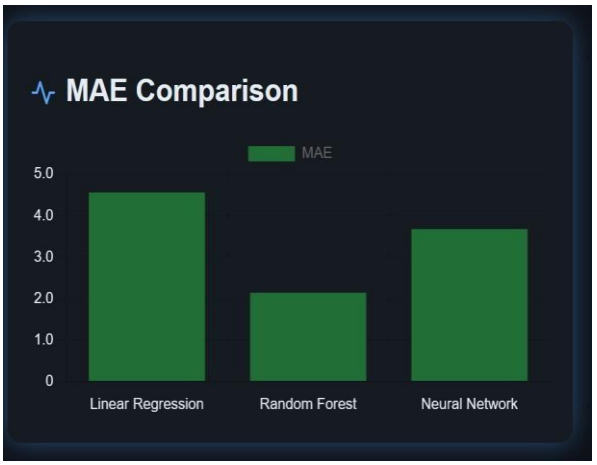


Fig. [8] MAE Comparison

The empirical findings showed that the Random Forest had the most desirable balance between accuracy and the robustness. Linear Regression was also a valuable starting point since it was simple to use, but it was not as effective in modeling non-linear relationships between distance, weather, and rider availability, as well as between distance and order attributes. The Neural Network was able to successfully model more complex patterns than Linear Regression, but its results were more volatile and sensitive to the parameter settings. The results, therefore, indicate the use of the Random Forest because it is the most suitable model to use in this data, due to its better performance in the variability of the real world. In this project the random forest model performed well across all 3 matrix where reasonably low showing that the model was able to make accurate prediction in most cases the r square score was also strong indicating that the model captured the relationship between the input feature and delivery time effectively the evaluation was also showed that random forests had handled noisy data better than simpler model like linear regression and also neural network this confirmed that random forest for suitable choice for this type of problem Another part of revolution involved looking at feature importance random forest provides a way to measure which feature contribute the most to the school because it is more after event the mark step was to evaluate

	Feature	Importance
0	Distance (km)	0.4712
1	Peak Hour	0.1076
2	Number of Items	0.1
3	Weather: Storm	0.0938
4	Weather: Rainy	0.0743
5	Order Value	0.0447
6	Weekend	0.0425
7	Riders Available	0.0334
8	Order Hour	0.0261
9	Weather: Fog	0.0046

Table 2: Feature Importance

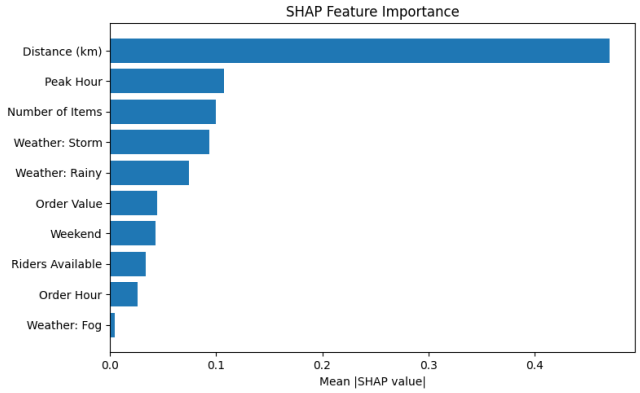


Fig. [9] Feature Importance

AWS model is useful and can be used easily and reliably in real world situations the deployment involves several AWS services including S3 Lambda API gateway and cloud watch the evaluation here focused on speed reliability and user experience. The first thing tested was the response time of the Lambda function says the model is loaded from South 3 and exhibited inside Lambda it was important to check how quickly the system could return a prediction the next was the API gateway tested to ensure that it correctly forwarded user inputs to the Lambda function and return the prediction without errors the API works smoothly and the evolution show that the system could handle different type of input as long as they follow the expected format the dashboard was also evaluated to see how well it display prediction and performance matrix the dashboard was easy to use and the charts help visualize the model accuracy user could enter the delivery details and instantly see the prediction time.

The overall system stability was evaluated using the AWS cloud watch logs led monitor to check for errors slow responses or unusual behavior the system remains stable throughout the testing period and no major issues were found this showed that the abyss deployment was reliable and suitable for real world use.

A manual error analysis was performed to explain further model behavior. This is a qualitative evaluation that is necessary since aggregate measures do not tell areas of model success or weaknesses. It was found that the bigger errors were mostly made in case of difficult delivery conditions, especially during periods of high delivery times or where several unfavorable conditions, like long distance, bad weather, and operational limitations, were combined. These models at times underestimated the real delivery times. The Actual and Predicted scatterplot showed that most predictions were close to the diagonal, which meant that the model was able to generalize in data pattern. However, extreme cases that exist in the periphery of a main cluster depicted reduced precision. This observation implies that the system can be satisfactorily maintained in a typical delivery condition but the cases with few or too late deliveries are more difficult to predict.

These were supported by a comparison of models. Linear Regression errors increased whenever the input variable-delivery time relationship became more complex. Even though, in some cases, the Neural Network performed better than the linear baseline, it was not that consistent as the Random Forest. Random Forest had the lowest overall number of substantial errors, which could be explained by its improved ability to model non-linear relationships, as well as interaction of features. As a result, Random Forest was the best candidate of this paper even though each model has its limitations when dealing with very aberrant delivery conditions.

Case	Distance	Items	Weather	Actual	Predicted	Abs. Er
5268	5.11	9	Clear	27.31	24.41	2.9
510	6.58	15	Clear	37.33	35.79	1.54
4328	5.03	6	Rainy	32.16	29.62	2.54
8380	2.05	2	Clear	13.92	15.56	1.64
9034	2.68	4	Clear	18.93	15.85	3.08

Table 3: Actual vs Predicted Delivery Time

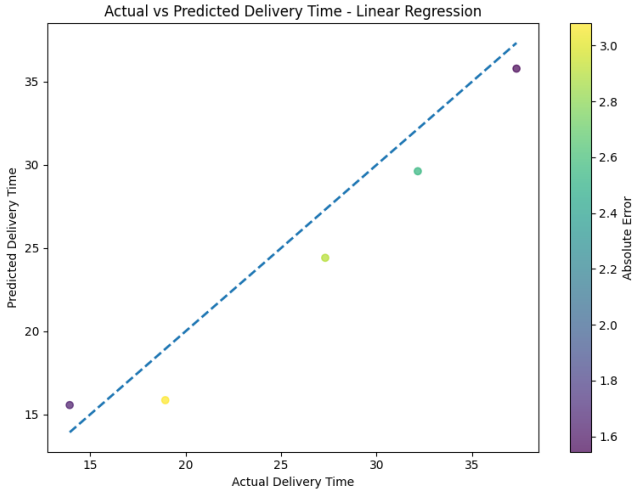


Fig. [10] Actual Vs Predicted Delivery Time-LR

Besides the accuracy of models, the implemented system was considered in relation to practical usability. AWS Lambda, API Gateway, and the web dashboard were used to test the prediction service to ensure that the end-to-end pipeline was working properly. The system was able to give predictions successfully when there is a valid input of the user and when the system is tested again. Those errors and abnormal behavior of running were observed with the help of CloudWatch logs. This test revealed that the implemented solution could be trusted in terms of interactive application and was appropriate as a prototype cloud-based prediction service.

V. CONCLUSION

The current study examined estimating the online grocery delivery times using machine learning techniques such as Random Forest, Neural Network and Linear Regression. The main objective was to build a scalable data pipeline in a cloud system and create predictive models that can estimate delivery time depending on the operational and environmental factors, such as the weather, geographic distance, traffic congestion, and order-specific factors. To achieve this goal, an elaborate data processing and machine learning pipeline was built based on cloud-based services. Raw delivery records were stored and go through a structured pipeline containing data cleansing, preprocessing, feature engineering operation.

Several machine learning models were trained and tested on the prediction task including Linear Regression, Random Forest Regression and Neural Network architecture. The experimental results showed that most models like Random Forest tend to be better performing than the basic algorithms like Linear Regression. Neural networks resulted in competitive results by modeling non-linear interrelations between the predictors and the delivery time. Model performance was measured based on common regression measures, such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and the coefficient of determination (R2).

In summary, this project shows how combining machine learning methods with cloud-based infrastructure can be used to optimize logistic operations, e.g. improving efficiency of deliveries and giving more accurate estimates of delivery times to consumers.

VI. FUTURE WORK

Although the current solution is successful in analyzing delivery time based on historical delivery data, multiple improvements and extensions can be made for further improve the performance and real-world situation for the solution. One potential improvement is using real-time data sources, such as live traffic conditions, weather API and courier availability. These dynamic inputs may be important to increase predictive accuracy as delivery time is usually determined by varying environmental conditions.

Another extension would be to play around with some very complex versions of machine learning and deep learning models such as Gradient Boosting algorithms (e.g. XGBoost or LightGBM) and recurrent neural networks that are designed for temporary data. These models would be able to capture more complex relationships in the dataset and potentially perform better than the current models.

Additionally, establishing automatic model training pipelines would enable the system to keep updating the predictions with the available new delivery data. This would ensure that the models would continue to be accurate over time and adaptable to changes in operational patterns. From a deploy ability perspective, the system could also be extended towards supporting real-time prediction services via available scalable APIs with production delivery platforms. Performance and data drift monitoring systems of the models can also contribute to the reliability of the systems. Lastly, the work in the future can focus on bigger and more diverse data sets such as in city or delivery apps, which would help to better predictive model and increase the robustness of the system to real-world applications.

VII. CONTRIBUTION

Team Member	**Contribution Area**	**Description of Work Done**
Aatmaja	Frontend Dashboard, UI Design, Dataset Preparation and Report Writing	Designed and developed the complete web dashboard using HTML, CSS JavaScript, chart.jS and AOS animation. Created a customised data set according to the project requirement.
Mrunal	AWS Architecture, Model Development AWS Glue Pipeline and Report writing	Design the cloud architecture using AWS services like S3, Lambda, API Gateway, Glue and cloud watch.
Vishal Verma	Documentation, Testing, Evaluating and Report Writing	Prediction accuracy by comparing a model output with expected delivery patterns. Organized the delivery data set

		used for model training. Build AWS Glue ETL jobs to transform load data into clean structure and data suitable for machine learning.
Meenakshi	Algorithms, Troubleshoot, Fixed API errors, Report Writing.	Built the models from the scratch. Troubleshoot deployment issues fixed API errors and ensure the system worked end to end.

Table 4: Contribution Table

VIII. REFRENCES

[1] Amazon Web Services, "Amazon S3 Documentation," 2024. [Online]. Available: https://docs.aws.amazon.com/s3/

[2] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/lambda/

[3] Amazon Web Services, "Amazon API Gateway Documentation," 2024. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[4] L. Breiman, "Random forests," Machine Learning, 2001. [Online]. Available: https://link.springer.com/article/10.1023/A:1010933404324

[5] Scikit-Learn, Regression Model, 2019. [Online]. Available: https://scikit-learn.org/stable/auto_examples/model_selection/index.html

[6] GeeksforGeeks, "Neural Networks: A Beginner's Guide," GeeksforGeeks, 2024. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/.

[7] GeeksforGeeks, "Linear Regression in Machine Learning," GeeksforGeeks, 2024. [Online]. Available: https://www.geeksforgeeks.org/linear-regression-in- machine-learning/.

[8] A. A. Alzahrani, "Predicting delivery time in last-mile logistics using machine learning techniques," M.S. thesis, Rochester Institute of Technology, Rochester, NY,USA,2023.[Online]. Available: https://repository.rit.edu/cgi/viewcontent.cgi?article=13604&context=theses

Jan Wolter, Thomas Hanne, "Prediction of service time for home delivery services using machine learning," ResearchGate, 2023. [Online]. Available: https://www.researchgate.net/publication/374292417_Prediction_of_service_time_for_home_delivery_services_using_machine_learning.